
2 0 2 3

산학협력프로젝트 - 인텔리즈

하계 근로 기간 문제 해결 과정과 2학기 개발 예정 내역

2022311466 김채리

Contents

01

프로젝트 개요

'CAVIAR-M'은 무엇이며, 어떤 이유로
개발되었는가?

02

상세 개발 내역 및 문제 해결 과정

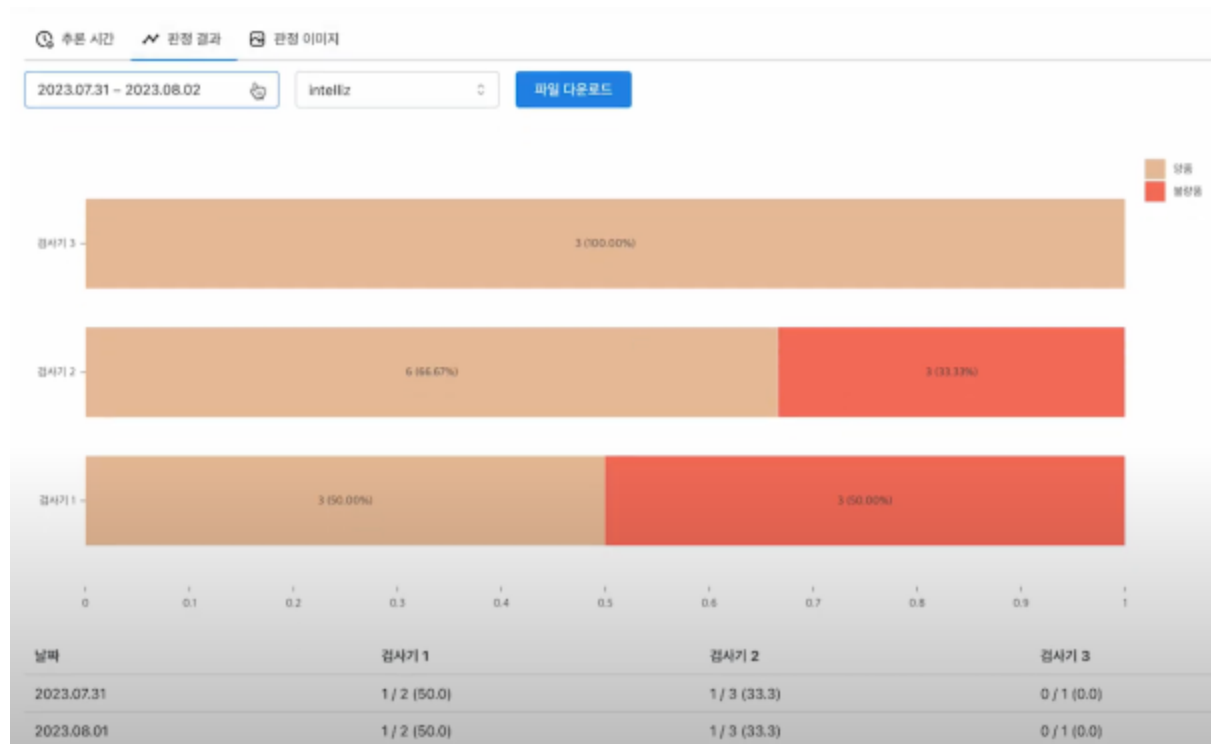
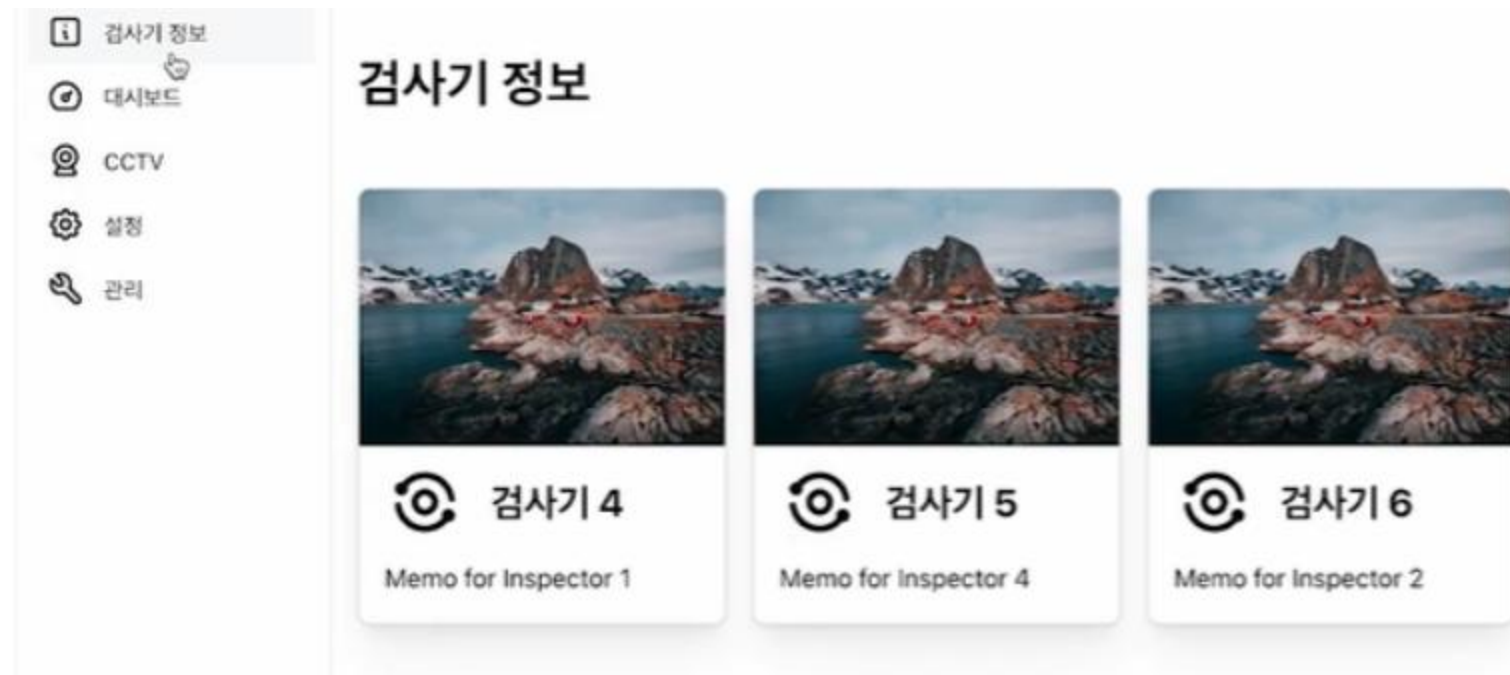
하계 근로 기간 동안 개발한 내용은 무엇이며
어떻게 문제를 해결하였는가?

03

스터디 및 2학기 개발 내역

앞으로 개발할 내용은 무엇이며
무엇을 준비했는가?

01. 프로젝트 개요



문제점

인텔리즈는 제품의 불량 여부를 판단하는 검사기의 소프트웨어와 하드웨어를 다른 공장 현장에 제공하는 기업입니다. 그러나 인텔리즈가 제공한 검사기에 문제나 변경 사항이 발생하게 되면, 검사기가 설치된 사이트에 직접 방문하여 이를 해결해야 합니다. 현장 방문 시간을 절약하고, 인텔리즈 본사에서 여러 검사기를 한번에 관리할 수 있도록 주어진 과제가 바로 'CAVIAR-M'입니다.

주요기능

CAVIAR-M은 여러 기능을 가진 웹 서비스입니다.

- i) 실시간으로 검사기 상태 조회 및 변경
- ii) 검사기에서 도출된 데이터 결과값 조회 / 시각화된 자료(이미지, 통계 자료) 열람
- iii) 고객사/회원 추가 및 관리
- iv) 검사기 CCTV 확인

기대효과

인텔리즈는 본 과제 결과물을 통해 검사기 관리의 시공간적 제약을 극복할 수 있으며, 검사기 도출 데이터의 효과적인 분석을 통해 자체 소프트웨어 기술을 발전시킬 수 있습니다.

고객사에게도 본 웹서비스가 배포되는 경우, 회원 기능을 통해 기존보다 고객을 쉽게 유치할 수 있으며, 검사기 원격 관리가 가능하다는 점에서 사용자의 서비스에 대한 만족도도 향상할 수 있습니다.



02. 상세 개발 내역 및 문제 해결 과정

CAVIAR-M 프로젝트의 백엔드 개발 중
발생한 문제 해결 과정과 주요 개발 내용을 소개합니다.

협업을 위한 문제 해결 과정

*migration: Django에서 모델 변경 사항을 스키마에 적용함.

Code Formatter

flake8와 black 도입을 통한 pre-commit과 clean code style 실천

모두 Python 프로그래밍 언어를 사용할 시 코드 품질과 스타일을 향상시키는 도구입니다. branch를 commit 하기 전에 자동으로 실행되어 코드의 에러를 감지하고 이를 수정하여 줍니다. 특히 두명 이상의 개발자가 작업 시 각기 다른 코드 스타일을 통일시켜 주는 역할을 합니다. 특정 기능을 추가하거나 삭제하고 싶다면 부가적인 설정을 통해 수정 가능합니다.

*Flake8: 에러가 발생하기 쉽거나 정해진 규격에 어긋나는 코드에 flag(표시)를 달아 놓는 기능.

*Black: flake8가 남긴 flag들을 감지하여 자동으로 수정해 주는 역할 수행.

ex) single(')/double("") qutoation 통일
한 라인의 글자 수/indent 및 enter 지점 통일

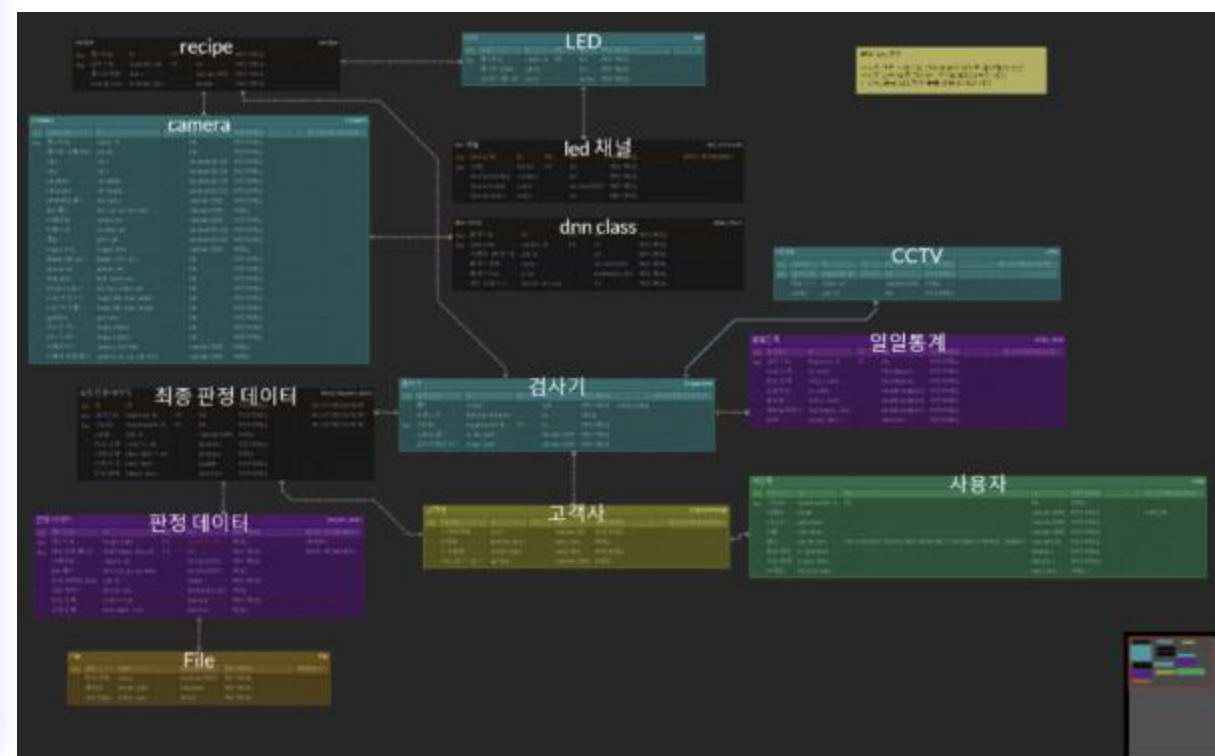
All done! ✨ 🍷 ✨
1 file reformatted, 1 file left unchanged.

API 문서

URL Pattern	Method	API name	Auth	리소스
v1/auth	POST	login	ALL	auth
v1/auth/logout	POST	logout	CUSTOMER_NORMAL	auth
v1/auth/pw_reset	POST	sendEmail	ALL	auth
v1/auth/pw_reset/key	POST	resetPw	ALL	auth
v1/auth/refresh	POST	tokenRefresh	CUSTOMER_NORMAL	auth
v1/users	POST	createUser	ALL	user
v1/users/id	GET	getUser	CUSTOMER_SUPER	user
v1/users	GET	getUsers	CUSTOMER_SUPER	user
v1/users/id	PATCH	updateUser	CUSTOMER_SUPER	user

MQTT

Topic Pattern	Message	published by
/settings/inspectorid/	Update Settings	server
/image/inspectorid/	Request Image	server
/settings/	Upload Settings	inspector
/inspec_result/	Upload Inspec Result	inspector
/inspec_state/	Update Inspector State	inspector
/inspec_state/inspectorid/	Check Inspector Storage	server



사전 작업 / 실시간 변경 문서

노선을 통한 api 및 mqtt 문서 작업, erdcloud를 통한 DB schema 정리

실시간으로 변경 및 조회가 가능한 노선과 erdcloud와 같은 웹서비스를 통해 수정된 사항을 빠르게 반영할 수 있도록 하였습니다. 특히 프로젝트 수행 기간 동안 기업 측에서 제공되는 schema가 자주 변경되어 같은 데이터베이스를 공유할 때 수정이 되면 오류가 발생하고, push 후 migration 파일이 도입되지 않아 충돌하는 경우가 빈번하게 발생하였는데, erdcloud를 통해 변경사항을 인지하고 필요한 모델 수정이나 migration 파일 파악을 할 수 있었습니다. api와 mqtt 문서 역시 요청 및 응답 형식에 수정사항이 있을 시 메모를 남기고, 개발 완료된 api에 체크를 남겨 현재까지의 개발 사항을 알기 쉽게 하였습니다. 이 외에도 gitlab에서의 branch명 통일과 issue 발행, slack 사용 등을 통해 효율을 높였습니다.

작동 / 유지 보수를 위한 문제 해결 과정

Exception

문제 상황 예측 및 대응

문제가 발생하였을 때 왜 문제가 발생하였고, 어떻게 해결할 수 있는지 공지하고자 Exception을 직접 만들어 적용하였습니다.

Camel Case 변환

react와 DRF의 json 호환

프론트엔드에 사용하는 JavaScript는 Camel Case가 적용된 반면, Django는 Snake Case가 적용되어 있기 때문에 문제가 발생합니다. Django의 json output형식을 Camel Case로 변경하여 응답하도록 구현하였습니다.

*Snake Case: ex) snake_case
Camel Case: ex) SnakeCase

Serializer 분리

코드 분배를 통한 clean-code style 유지

한 모델을 만들게 될 때, api의 기능을 관리하는 대부분의 기능을 view.py에서 구현합니다. 이 때 권한 제한, 입력 오류 등의 Exception, swagger 세팅, mqtt 메시지 전송 등의 기능을 추가하다보면 view.py가 난잡해지게 됩니다. 이를 방지해 코드의 가독성을 높이고, 수정사항을 쉽게 반영할 수 있도록 serializer를 여러개로 분리하여 만들어 view.py에서 이를 호출하여 사용할 수 있는 형식을 변경하였습니다.

```
FinalInspecDataListResponseSerializer.py
FinalInspecDataQuerySerializer.py
FinalInspecDataSerializer.py
InspectorMemoSerializer.py
InspectorResponseSerializer.py
InspectorSerializer.py
InspectorsResponseSerializer.py
InspectorStateSerializer.py
OverallStatListResponseSerializer.py
OverallStatQuerySerializer.py
OverallStatSerializer.py
SettingResponseSerializer.py
UpdateInspectorSerializer.py
```

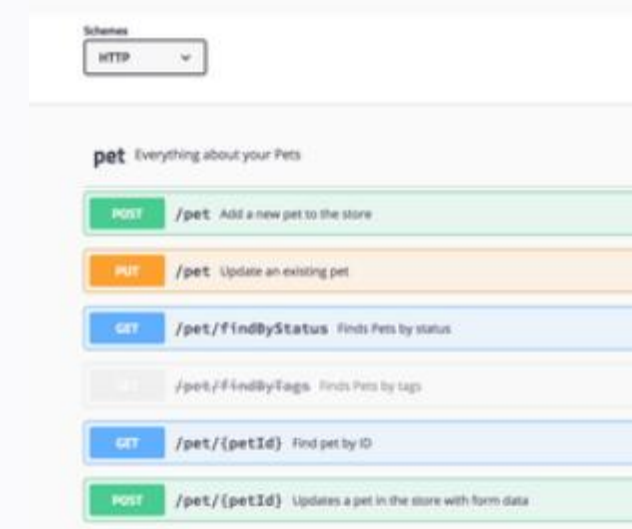
*serializer: django에 저장되어 있는 모델 인스턴스를 REST API에서 사용하는 JSON의 형태로 바꿔주는 역할 수행.

Swagger

Swagger를 통한 인수인계 자료 준비

Swagger는 API를 설계, 빌드, 문서화 및 사용하는 데 도움이 되는 오픈 소스 도구로, Django 코드 내에서 추가가 가능합니다. 이를 추가하여 api를 정리하고, api 사용 방법에 대한 설명을 제공하였습니다.

- i) API에 들어가는 parameter 정의
- ii) 응답 및 요청 형식 정리
- iii) API name과 model tag 부여
- iv) 각 API의 description 추가



API 개발 주요 문제 해결 과정

File API

파일을 받아서 특정 공간 내에 저장하는 `upload_file()`과 파일을 가져오는 `get_file()` API를 개발하였습니다.

uuid 사용

파일 이미지 저장 시 파일 이름이 겹치는 경우를 방지하고 이미지 조회 시 url 주소에 고유한 데이터를 넣기 위해, `uuid`를 사용하였습니다.

tar.gz 파일

`tar.gz` 파일을 업로드하여, 바로 비교하여 저장하는 데 어려움이 있었습니다. 특히 `macOS` 환경에서 만들어진 `tar.gz`파일은 메타데이터 파일이 함께 생성되어 만들어지기 때문에 이미지 파일만을 업로드하기 쉽지 않았습니다. `tar.gz` 폴더를 압축 해제하여 특정 이름으로 시작하는 파일(._)이나 주어진 확장자 외 파일은 삭제하도록 설정하였습니다.

MQTT

검사기가 업데이트할 값을 전송하고, 서버가 검사기에 경고를 전송하는 데 사용된 기능입니다.

serializer 사용

서버에서 설정 페이지에 있는 검사기, CCTV, 카메라 등의 세부사항이 수정될 때마다 검사기에 발송되는 메시지를 일일이 작성하기에는 그 양이 방대하였습니다. `serializer`를 여러개 합쳐서 구현함으로써 코드를 효율적으로 작성할 수 있었습니다.

*`uuid`: 절대로 겹치지 않는 고유한 식별자로, 128비트(16바이트) 길이의 숫자로 구성

*`mqtt`: 경량의 Publish/Subscribe(Pub/Sub) 메시징 프로토콜; IoT를 위해서 낮은 전력, 낮은 대역폭 환경에서도 사용할 수 있도록 설계



03. 스터디 진행 사항 및 2학기 개발 내역

하계 방학 기간 동안 2학기 개발을 위해 진행하였던 스터디
내용과 2학기에 추가적으로 개발하게 될 기능을 소개합니다.

VAD/YOLO 스터디

VAD 논문 스터디

Video Anomaly Detection의 약자로 영상에서 미리 학습된 이상현상을 탐지하는 기술로, 본 논문에서 소개된 VAD는 세 단계를 거칩니다.

i) Preprocessing - 프레임 단위로 움직임 추출 / 프레임 단위로 물체 검출

ii) Feature Extraction - 추출한 움직임을 통해 속력 파악 / 검출한 물체로 포즈 파악

iii) Density Estimation - 추출한 정보를 바탕으로 이상 점수를 계산

Attribute-based Representations for
Accurate and Interpretable Video Anomaly Detection

Tal Reiss, Yedid Hoshen
School of Computer Science and Engineering
The Hebrew University of Jerusalem, Israel

YOLO 실습

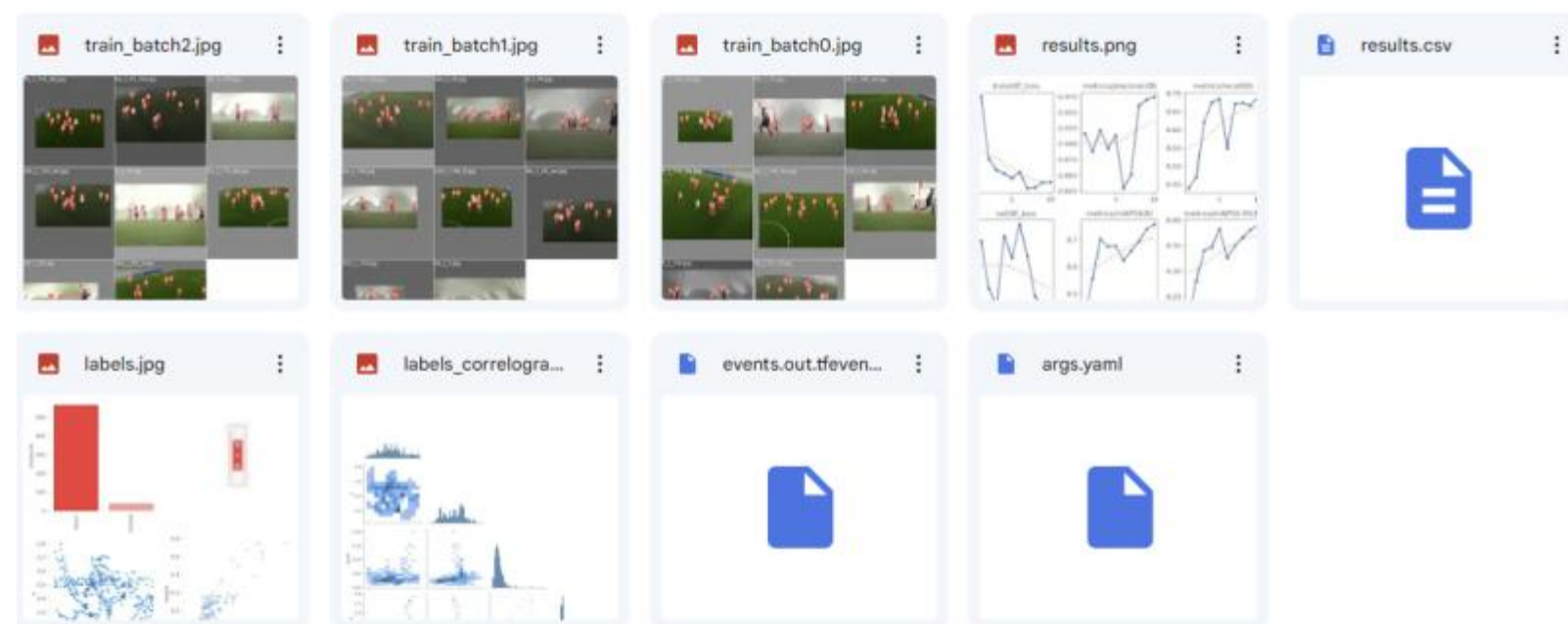
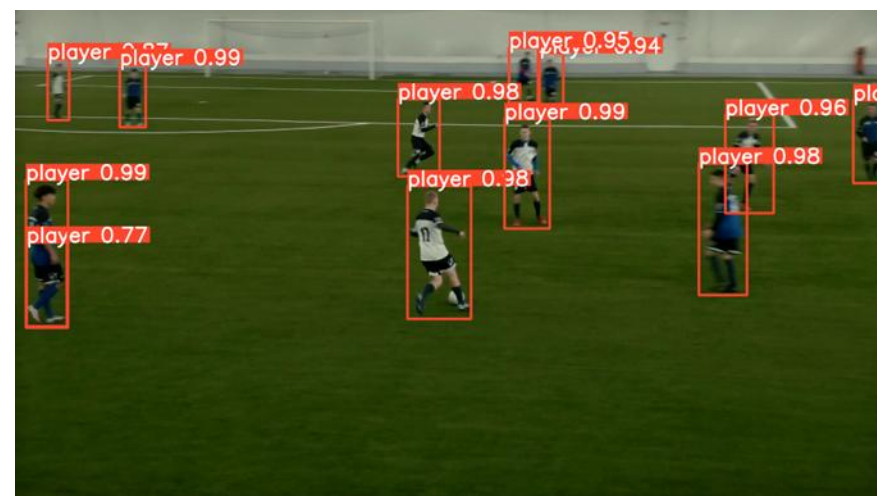
You Only Look Once의 약자로 VAD의 물체 검출 단계에 쓰이는 기술의 일종입니다.

i) Data Annotation - 인식할 물체의 사진에 표시하면, 각 지점을 기록한 txt 파일이 형성됩니다.

ii) Yolo v8 학습 - 원하는 버전을 골라 해당 폴더 내 물체 사진을 학습시킵니다.



```
0 0.097135 0.273611 0.026042 0.091667
0 0.204427 0.433333 0.033333 0.113889
0 0.260547 0.300463 0.034635 0.113889
0 0.332031 0.169676 0.018750 0.082870
0 0.349349 0.197685 0.026823 0.088889
0 0.464193 0.276620 0.024219 0.099537
0 0.553646 0.292593 0.033333 0.091667
0 0.530990 0.366667 0.033333 0.092593
0 0.496484 0.409491 0.027344 0.085648
0 0.505599 0.530556 0.037240 0.113889
0 0.650130 0.478935 0.040885 0.119907
0 0.735547 0.183796 0.024740 0.071296
0 0.786589 0.156019 0.032031 0.072222
0 0.938021 0.240972 0.026042 0.078241
0 0.979688 0.334259 0.029167 0.096296
1 0.579297 0.332870 0.008594 0.017593
```



현재 작업 중인 2학기 개발 사항

현재 역할 분배 후 개발 중에 있거나 완료한 추가 개발사항입니다.

검사기 상태

스토리지, 데이터
등을 mqtt로 변경하고
공간 부족 시 알림
전송하는 기능 추가

회원가입 수정

회사 key 없이도
회원이 가입이
가능하도록 변경

데모 페이지 추가

회원이 아닌 접속자가
체험할 수 있는
데모 페이지 추가

코드 안정화

작업 중 발생하는
에러 해결 및 clean code
유지 노력



VAD 적용

제품 불량 여부를 검사하는 회사 소프트웨어에 VAD 기능을 추가하여
정확성을 높이는 작업을 진행할 예정입니다.

감사합니다.
